# WEB TECHNOLOGY LAB

# Practical Record – Questions 1 to 11

Easy-to-write programs with Aim, Algorithm, Program, Output and Result

### Topics covered

- JavaScript Classes and Functions
- Bootstrap Grid, Cards and Badges
- Responsive Design and CSS Media Queries
- JavaScript Modules and Asynchronous Programming
- Infosys Airlines, WeCare and XYZPharma mini-projects

**Important note:** These are simplified standalone practice versions based on the questions shared. The final-project lab guide may require a supplied service/API, specific screens, role-based actions or routing. Follow your instructor's guide if it differs.

# 1. Exercise on Classes – Product Class

**Aim:** To create a Product class using a constructor and methods, create two objects, and display their details.

## Algorithm:

1. Start the program.
2. Create a class named Product.
3. Define a constructor with name, price and quantity.
4. Create a method to display product details.
5. Create two objects and call the display method.
6. Display the output and stop.

## Program:

**File:** Q1_Product.html

```
<!DOCTYPE html>
<html>
<head><title>Product Class</title></head>
<body>
<h2>Product Details</h2>
<script>
class Product {
    constructor(name, price, quantity) {
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }
    display() {
        document.write("Product Name: " + this.name + "<br>");
        document.write("Price: " + this.price + "<br>");
        document.write("Quantity: " + this.quantity + "<br>");
        document.write("------------------<br>");
    }
}
let p1 = new Product("Laptop", 50000, 2);
let p2 = new Product("Mobile", 15000, 3);
p1.display();
p2.display();
</script>
</body>
</html>
```

## Output:

```
Product Details
Product Name: Laptop
Price: 50000
Quantity: 2
------------------
Product Name: Mobile
Price: 15000
Quantity: 3
------------------
```

**Result:** Thus, the program was successfully executed using a Product class, constructor and methods to create two objects and display their details.

# 2. Functions – Shopping Cart Total

**Aim:** To calculate a shopping cart total using JavaScript functions and test normal and boundary values.

## Algorithm:

1. Start the program.
2. Create calculateTotal() function.
3. Pass price and quantity as arguments.
4. Multiply price by quantity and return the total.
5. Test normal and boundary values.
6. Display the results and stop.

## Program:

**File:** Q2_Functions.html

```
<!DOCTYPE html>
<html>
<head><title>Shopping Cart</title></head>
<body>
<h2>Shopping Cart Total</h2>
<script>
function calculateTotal(price, quantity) {
    return price * quantity;
}
let total1 = calculateTotal(100, 2);
let total2 = calculateTotal(500, 3);
let total3 = calculateTotal(0, 5);
let total4 = calculateTotal(100, 0);

document.write("Product 1: Rs." + total1 + "<br>");
document.write("Product 2: Rs." + total2 + "<br>");
document.write("Zero Price: Rs." + total3 + "<br>");
document.write("Zero Quantity: Rs." + total4);
</script>
</body>
</html>
```

## Output:

```
Shopping Cart Total
Product 1: Rs.200
Product 2: Rs.1500
Zero Price: Rs.0
Zero Quantity: Rs.0
```

**Result:** Thus, the program was successfully executed using functions to calculate and display the shopping cart total.

# 3. Grid System – Bootstrap

**Aim:** To create a responsive webpage using the Bootstrap grid system with rows and columns.

## Algorithm:

1. Start the program.
2. Create an HTML page and add the Bootstrap CSS link.
3. Create a container and row.
4. Add three columns.
5. Display the content in a browser and stop.

## Program:

**File:** Q3_Grid.html

```
<!DOCTYPE html>
<html>
<head>
<title>Bootstrap Grid</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
<div class="container">
    <h2>Bootstrap Grid System</h2>
    <div class="row">
        <div class="col bg-primary text-white">Column 1</div>
        <div class="col bg-success text-white">Column 2</div>
        <div class="col bg-danger text-white">Column 3</div>
    </div>
</div>
</body>
</html>
```

## Output:

```
Bootstrap Grid System
[ Column 1 ]  [ Column 2 ]  [ Column 3 ]
Desktop: Three columns appear in one row.
Mobile: Columns adjust to the available screen width.
```

**Result:** Thus, the responsive webpage was successfully created using the Bootstrap grid system.

# 4. Badges, Jumbotron and Cards

**Aim:** To create a product webpage using Bootstrap cards and badges to display product details.

## Algorithm:

1. Start the program.

2. Create an HTML page and add Bootstrap CSS.

3. Create a heading and product cards.

4. Add badges to the cards.

5. Display the webpage and stop.

## Program:

**File:** Q4_Cards.html

```
<!DOCTYPE html>
<html>
<head>
<title>Product Cards</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
<div class="container mt-4">
    <h2 class="text-center">Our Products</h2>
    <div class="row">
        <div class="col">
            <div class="card p-3">
                <h3>Laptop</h3>
                <span class="badge bg-success">Available</span>
                <p>Price: Rs.50000</p>
                <button class="btn btn-primary">Buy Now</button>
            </div>
        </div>
        <div class="col">
            <div class="card p-3">
                <h3>Mobile</h3>
                <span class="badge bg-danger">Limited Stock</span>
                <p>Price: Rs.15000</p>
                <button class="btn btn-primary">Buy Now</button>
            </div>
        </div>
    </div>
</div>
</body>
</html>
```

## Output:

```
Our Products
Laptop
Available
Price: Rs.50000
[Buy Now]

Mobile
Limited Stock
Price: Rs.15000
[Buy Now]
```

**Result:** Thus, the product webpage was created using Bootstrap cards and badges. Note: Jumbotron was removed in Bootstrap 5; a styled container can be used instead.

# 5. Responsive Design and Patterns

**Aim:** To create a responsive webpage using HTML and CSS Flexbox with a navigation bar and content section.

## Algorithm:

1. Start the program.
2. Create an HTML page and navigation bar.
3. Create three content boxes.
4. Apply CSS Flexbox styling.
5. Use a media query for mobile screens.
6. Display the webpage and stop.

## Program:

**File:** Q5_Responsive.html

```
<!DOCTYPE html>
<html>
<head>
<title>Responsive Website</title>
<style>
body { font-family: Arial; }
nav { background: blue; color: white; padding: 15px; }
.container { display: flex; }
.box {
    background: lightblue;
    padding: 30px;
    margin: 10px;
    flex: 1;
}
@media (max-width: 600px) {
    .container { flex-direction: column; }
}
</style>
</head>
<body>
<nav><h2>My Website</h2></nav>
<h2>Welcome</h2>
<div class="container">
    <div class="box">HTML</div>
    <div class="box">CSS</div>
    <div class="box">JavaScript</div>
</div>
</body>
</html>
```

## Output:

```
My Website
Welcome

Desktop:
[ HTML ]  [ CSS ]  [ JavaScript ]

Mobile:
[ HTML ]
[ CSS ]
[ JavaScript ]
```

**Result:** Thus, the responsive webpage was created using Flexbox and CSS media queries.

# 6. Modular Programming – Export and Import

**Aim:** To create JavaScript modules using export and import and execute them using a local development server.

## Algorithm:

1. Start the program.
2. Create math.js and export a function.
3. Create main.js and import the function.
4. Create index.html and load main.js as a module.
5. Run index.html using a local server and stop.

## Program:

**File:** Create a folder Q6 containing index.html, math.js and main.js.

```
FILE 1: math.js
----------------
export function add(a, b) {
    return a + b;
}

FILE 2: main.js
---------------
import { add } from './math.js';
let result = add(10, 20);
document.getElementById("output").innerHTML =
    "Addition = " + result;

FILE 3: index.html
-----------------
<!DOCTYPE html>
<html>
<head><title>Modular Programming</title></head>
<body>
<h2>JavaScript Modules</h2>
<p id="output"></p>
<script type="module" src="main.js"></script>
</body>
</html>
```

## Output:

```
JavaScript Modules
Addition = 30

Execution:
Open the Q6 folder in VS Code.
Right-click index.html and select
"Open with Live Server".
```

**Result:** Thus, JavaScript modules were created using export and import and executed using a local development server.

# 7. Statements and Expressions – Student Mark Calculation

**Aim:** To use variables, operators, conditional statements and loops to calculate and validate student marks.

## Algorithm:

1. Start the program.
2. Create an array to store marks.
3. Use a loop to calculate total marks.
4. Use if-else to validate the total.
5. Display marks and result, then stop.

## Program:

**File:** Q7_Statements.html

```
<!DOCTYPE html>
<html>
<head><title>Statements and Expressions</title></head>
<body>
<h2>Student Mark Calculation</h2>
<div id="output"></div>
<script>
let marks = [80, 70, 90];
let total = 0;

for (let i = 0; i < marks.length; i++) {
    total = total + marks[i];
}

let result = "";
if (total <= 300 && total >= 0) {
    result = "Valid Marks";
} else {
    result = "Invalid Marks";
}

document.getElementById("output").innerHTML =
    "Mark 1: " + marks[0] + "<br>" +
    "Mark 2: " + marks[1] + "<br>" +
    "Mark 3: " + marks[2] + "<br>" +
    "Total: " + total + "<br>" +
    "Result: " + result;
</script>
</body>
</html>
```

## Output:

```
Student Mark Calculation
Mark 1: 80
Mark 2: 70
Mark 3: 90
Total: 240
Result: Valid Marks
```

**Result:** Thus, the program was executed using variables, operators, conditional statements and loops to calculate and validate student marks.

# 8. Asynchronous Programming – Async and Await

**Aim:** To use async and await to handle an asynchronous operation and display loading, success and error messages.

## Algorithm:

1. Start the program.
2. Create an asynchronous function.
3. Display a loading message.
4. Use await to wait for an operation.
5. Display the success message.
6. Handle errors using try-catch and stop.

## Program:

**File:** Q8_Async.html

```
<!DOCTYPE html>
<html>
<head><title>Asynchronous Programming</title></head>
<body>
<h2>Async and Await Example</h2>
<p id="output">Click the button</p>
<button onclick="getData()">Load Data</button>
<script>
async function getData() {
    document.getElementById("output").innerHTML = "Loading...";
    try {
        await new Promise(resolve =>
            setTimeout(resolve, 2000)
        );
        document.getElementById("output").innerHTML =
            "Data Loaded Successfully!";
    }
    catch (error) {
        document.getElementById("output").innerHTML =
            "Error Loading Data!";
    }
}
</script>
</body>
</html>
```

## Output:

```
Async and Await Example
Before clicking: Click the button
After clicking: Loading...
After 2 seconds: Data Loaded Successfully!
```

**Result:** Thus, the program was executed using async and await to handle an asynchronous operation and display loading and success messages.

# 9. Infosys Airlines – Flight Booking Application

**Aim:** To develop a flight booking application to book seats, check seat availability, view bookings and delete bookings.

## Algorithm:

1. Start the program.
2. Create a flight booking webpage.
3. Enter passenger name and destination.
4. Book a seat if seats are available.
5. Display current seat availability and bookings.
6. Allow the user to delete a booking.
7. Stop the program.

## Program:

**File:** Q9_Airlines.html

```
<!DOCTYPE html>
<html>
<head><title>Airline Booking</title></head>
<body>
<h2>Infosys Airlines</h2>
<h3>Flight Booking</h3>

Passenger Name: <input type="text" id="name"><br><br>
Destination:
<select id="destination">
    <option>Chennai</option>
    <option>Delhi</option>
    <option>Mumbai</option>
</select><br><br>

<button onclick="book()">Book Flight</button>
<h3>Available Seats: <span id="seats">5</span></h3>
<h3>My Bookings</h3>
<div id="output"></div>

<script>
let seats = 5;
let bookings = [];

function book() {
    let name = document.getElementById("name").value;
    let place = document.getElementById("destination").value;

    if (name == "") {
        alert("Enter Passenger Name");
    } else if (seats > 0) {
        bookings.push(name + " - " + place);
        seats--;
        document.getElementById("seats").innerHTML = seats;
        display();
    } else {
        alert("No Seats Available");
    }
}

function display() {
    let result = "";
    for (let i = 0; i < bookings.length; i++) {
        result += bookings[i] +
          " <button onclick='removeBooking(" + i +
          ")'>Delete</button><br>";
    }
    document.getElementById("output").innerHTML = result;
}

function removeBooking(index) {
    bookings.splice(index, 1);
    seats++;
    document.getElementById("seats").innerHTML = seats;
    display();
}
</script>
</body>
</html>
```

## Output:

```
Infosys Airlines
Passenger Name: Vasanth
Destination: Chennai
```

```
Available Seats: 4

My Bookings:
Vasanth - Chennai
```

**Result:** Thus, the simplified flight booking application was developed to book flights, check seat availability, view bookings and delete bookings. The actual lab service/API may require additional integration.

# 10. WeCare – Healthcare Application

**Aim:** To develop a WeCare healthcare application with a patient registration form and input validation.

## Algorithm:

1. Start the program.
2. Create an HTML webpage and display WeCare heading.
3. Create patient name and age fields.
4. Validate the entered details.
5. Display patient information or an error message.
6. Stop the program.

## Program:

**File:** Q10_WeCare.html

```
<!DOCTYPE html>
<html>
<head>
<title>WeCare Healthcare</title>
<style>
body { font-family: Arial; text-align: center; }
h1 { background: blue; color: white; padding: 15px; }
input, button { padding: 10px; margin: 5px; }
</style>
</head>
<body>
<h1>WeCare Healthcare</h1>
<h2>Patient Registration</h2>

<input type="text" id="name" placeholder="Enter Name"><br>
<input type="number" id="age" placeholder="Enter Age"><br>
<button onclick="register()">Register</button>
<h3 id="result"></h3>

<script>
function register() {
    let name = document.getElementById("name").value;
    let age = document.getElementById("age").value;

    if (name == "" || age == "") {
        document.getElementById("result").innerHTML =
            "Please fill all details";
    } else if (age <= 0) {
        document.getElementById("result").innerHTML =
            "Enter valid age";
    } else {
        document.getElementById("result").innerHTML =
            "Registration Successful!<br>" +
            "Name: " + name + "<br>Age: " + age;
    }
}
</script>
</body>
</html>
```

## Output:

```
WeCare Healthcare
Patient Registration
Registration Successful!
Name: Vasanth
Age: 20
```

**Result:** Thus, the simplified WeCare application was created using HTML, CSS and JavaScript with patient registration and validation. The full lab guide may require additional screens and role-specific actions.

# 11. XYZPharma – Pharmaceutical Website

**Aim:** To develop an XYZPharma website with Home, About, Products, Product Details and Investors sections with navigation features.

## Algorithm:

1. Start the program.
2. Create an HTML webpage and navigation bar.
3. Create Home and About sections.
4. Display products and product details.
5. Create an Investors section.
6. Add navigation links and stop.

## Program:

**File:** Q11_XYZPharma.html

```
<!DOCTYPE html>
<html>
<head>
<title>XYZPharma</title>
<style>
body { font-family: Arial; margin: 0; text-align: center; }
nav { background: blue; padding: 15px; }
nav a { color: white; margin: 10px; text-decoration: none; }
section { padding: 30px; }
</style>
</head>
<body>
<h1>XYZPharma</h1>
<nav>
<a href="#home">Home</a>
<a href="#about">About</a>
<a href="#products">Products</a>
<a href="#details">Details</a>
<a href="#investors">Investors</a>
</nav>

<section id="home">
<h2>Welcome to XYZPharma</h2>
<p>Quality Healthcare for Everyone</p>
</section>

<section id="about">
<h2>About Us</h2>
<p>We provide quality medicines.</p>
</section>

<section id="products">
<h2>Our Products</h2>
<p>Paracetamol - Rs.20</p>
<p>Vitamin Tablets - Rs.50</p>
</section>

<section id="details">
<h2>Product Details</h2>
<p>Paracetamol is used to reduce fever.</p>
</section>

<section id="investors">
<h2>Investors</h2>
<p>Welcome to our investor section.</p>
</section>
</body>
</html>
```

## Output:

```
XYZPharma
Home – Welcome to XYZPharma
Quality Healthcare for Everyone

About Us
We provide quality medicines.

Our Products
Paracetamol - Rs.20
Vitamin Tablets - Rs.50

Product Details
```

```
Paracetamol is used to reduce fever.

Investors
Welcome to our investor section.
```

**Result:** Thus, the XYZPharma website was created using HTML and CSS with Home, About, Products, Product Details and Investors sections. A lab guide requiring application routing may need separate routes or pages.